Part 3 — Normative Loader, Runtime Assurance, and `UpdateCert`

Verifiable Automaton, TCB Hardening (STRIDE), and Uniform Receipts

SpecChain Formal Specification

September 17, 2025

Abstract

This Part specifies the Normative Loader as a verifiable automaton; fixes the trusted-computing-base (TCB) boundary with a boot-time attestation receipt; and formalizes dynamic adaptation via `UpdateCert` with *step-preserving bisimulation* (replay-impact equivalence) and Σ-*non-expansion* proof obligations. We state the *safety-or-halt* theorem and a No-Zeno liveness discipline using a well-founded lexicographic rank; baseline checks (I1–I10) produce uniform, byte-replayable receipts bound by `proof_surface` under `canon/v1` with `BLAKE3-256`. Acceptance is gated by per-axis budgets in $\mathbf{Q}$ (fail-closed unknowns = $\bot$) and, when enabled, by tail contracts (I11–I13) from Part 4; every successful or failing run emits auditable evidence suitable for deterministic third-party replay. The pipeline order $\mathsf{EF} \to \mathsf{DRO} \to \mathsf{PR} \to \mathsf{CBF} \to \Omega$ and seed discipline are harmonized with Part 1; notation and unknown semantics follow the absorbing-bottom convention.

1 Scope and Interfaces

This Part specifies the *Normative Loader*—the sole arbiter of runtime state transitions for the pipeline

$$\mathsf{EF} \to \mathsf{DRO} \to \mathsf{PR} \to \mathsf{CBF} \to \Omega,$$

and the enforcer of the *safety-or-halt* contract. All outcomes (PASS/FAIL) materialize as canonical JSON receipts bound into the run's `proof_surface`, enabling deterministic, third-party replay. The Loader is defined as a verifiable automaton (see Section 6) whose steps are gated by baseline checks (I1–I10, Section 3), a well-founded rank (Section 2), and a No-Zeno dwell discipline (Section 4). When enabled by policy, tail-contract hooks (I11–I13) from Part 4 gate the terminal ACCEPT.

1.1 Role of the Normative Loader

The Loader orchestrates stage adapters, applies typed guards, invokes pricing kernels, aggregates acceptance state $q \in \mathbf{Q}$, and compares q to budgets B axis-wise. It implements a *guard-first* discipline: frontier rows with unknown/missing priced coordinates are rejected *before* any pricing ($\bot$ is absorbing/fail-closed). Every step either strictly decreases the lexicographic rank (Section 2) or halts with an auditable `RejectionCert`.

1.2 Foundational Assumptions

The Loader's guarantees rely on a small set of cross-part assumptions; each is made explicit and bound by receipts.

Assumption 1.1 (Deterministic replay envelope)**.** All computations execute under a pinned `EnvLock/ReplayPlan` pair: floating-point modes/libraries, RNG algorithm, and serialization rules (`canon/v1`) are fixed. The digest of (`EnvLock`, `ReplayPlan`) is bound into `proof_surface`; compliant re-execution with identical inputs reproduces identical bytes.

Assumption 1.2 (Seed derivation discipline)**.** A per-run master seed is derived by a pure KDF (e.g., HKDF over `BLAKE3-256`) from `root_print`, the protocol `suite`, the lexicographically sorted parent digests, and the current `proof_surface`:

$$\texttt{seed} := \mathrm{HKDF}_{\texttt{BLAKE3-256}}(\texttt{root_print} \parallel \texttt{suite} \parallel \mathrm{sort}(\texttt{parents}) \parallel \texttt{proof_surface}).$$

Stage-local RNGs are derived via domain separation. No wall-clock or ambient entropy is consulted.

Assumption 1.3 (Acyclic evaluation order)**.** The Loader's evaluation order is acyclic: no stage induces a cyclic dependency or infinite regress; admissible evaluation trees respect adapter order and guards.

Assumption 1.4 (Typed adapter properties)**.** Each stage adapter declares an axis whitelist and behaves as a monotone homomorphism into $\mathbf{Q}$: (i) *Axis isolation*—outputs affect only declared axes; (ii) *Monotone homomorphism*—for inputs $x \le x'$, $A(x) \le_{\mathbf{Q}} A(x')$, and $A(x \cup y) = A(x) \wedge A(y)$ where defined. Violations are fail-closed with `RejectionCert`.

Invariant 1.5 (Guard-first, fail-closed unknowns)**.** Rows with $\bot$ on any priced coordinate are rejected *prior* to pricing; $\bot$ does not propagate into q. This aligns UI-level "missing" with algebraic bottom at the guard boundary.

1.3 Inter-part Interfaces

Interface to Part 2 (Algebra and Pricing). The Loader is a *client* of the Part 2 acceptance algebra and pricing contract: (i) all q-updates use the typed product-quantale operators (vertical $\wedge$, horizontal $\odot$); (ii) kernels are total, side-effect-free, and monotone over normalized, dimensionless inputs; and (iii) the axis catalog digest that defines $\mathbf{Q}$ and its uncertainty lift Q' is pinned in state and bound by `proof_surface`.

Interface to Part 4 (Termination/Tails). The ACCEPT transition is enabled only upon a valid tail-contract receipt (I11–I13), whose canonical witness bytes are included in the accepted block's evidence bundle.

Dynamic adaptation (Part 3 & cross-part). Updates are mediated by `UpdateCert` (see Section 8), requiring (a) step-preserving bisimulation for replay-impact equivalence and (b) Σ-non-expansion to prevent semantic obligation erasure. `UpdateCert` applies atomically; failure is fail-closed with `RejectionCert`.

TCB boundary (boot-time). At boot (rule T-Boot), the Loader emits an `AttestationReceipt` binding the loader binary hash, `EnvLock/ReplayPlan` descriptors, and identity-graph root; this receipt is included in the run's evidence bundle and thus in `proof_surface`.

2 Lexicographic Rank and Policy

Intent. The Loader's liveness is enforced by a strictly decreasing, well-founded rank. This section (i) states a *parametric* rank signature so future protocol versions can add components without redoing the proof, (ii) instantiates the canonical 5-tuple used in this Part, and (iii) defines the discretization that connects continuous frontier slack to an integer rank component.

Definition 2.1 (Rank signature (parametric))**.** A *rank signature* is a tuple $\mathcal{R} = (C_0, \ldots, C_{k-1})$ of sets equipped with well-founded relations $\prec_i$, together with a precedence order $C_0 \succ C_1 \succ \cdots \succ C_{k-1}$. Write $R \triangleq \prod_{i=0}^{k-1} C_i$ and let $\lhd_{\text{lex}}$ be the induced lexicographic order: $r' \lhd_{\text{lex}} r$ iff $\exists j.\ r'_j \prec_j r_j$ and $\forall i < j.\ r'_i = r_i$.

Remark 2.2 (Well-foundedness)**.** If each $(C_i, \prec_i)$ is well-founded and $k < \infty$, then $(R, \lhd_{\text{lex}})$ is well-founded. Consequently, any infinite sequence of strict decreases in $\lhd_{\text{lex}}$ is impossible.

Definition 2.3 (Loader rank (canonical instantiation))**.** The Loader maintains $\boldsymbol{r} \in \mathbb{N}^5$ ordered by $\lhd_{\text{lex}}$ with dominance $r_0 \succ r_1 \succ r_2 \succ r_3 \succ r_4$:

R1. **Invariant debt** r_0: the number of currently failing baseline checks (I1–I10).

R2. **Frontier slack** r_1: a discretized deficit-to-budget derived from Δ_{fr} (or a certified over-approximation $\widehat{\Delta}_{\text{fr}}$) and policy budgets B.

R3. **Adapter obligations** r_2: outstanding emissions required by adapter contracts (e.g., attestations, receipts promised by monotonicity tests).

R4. **Audit backlog** r_3: pending evidence bindings (hashes, signatures, indices) needed to satisfy I10.

R5. **Liveness dwell counter** r_4: remaining steps before the No-Zeno cap $T_{\max}$ forces a halt (see Section 4).

Invariant 2.4 (Dominance)**.** No transition may increase a more significant component to decrease a less significant one. Equivalently, if a step changes $\boldsymbol{r}$ from r to r', then either $r' = r$ or $r' \lhd_{\text{lex}} r$.

Discretizing frontier slack. Budgets B and debits Δ_{fr} live in the (generally continuous) acceptance algebra $\mathbf{Q}$. To connect real-valued progress to an integer rank:

Definition 2.5 (Slack discretization)**.** Let the (per-policy) minimal progress quantum be $\delta > 0$. Let deficit(g) be the nonnegative per-group deficit after applying the certified debit on group g, and let Deficit $\triangleq \sum_g$ deficit(g) be the policy-aggregated scalar. Set

$$r_1 \ := \ \left\lceil \frac{\max\{0,\ \text{Deficit}\}}{\delta} \right\rceil .$$

Lemma 2.6 (Monotonicity and strict decrease)**.** *If* Deficit *decreases by at least δ between successive states, then r_1 strictly decreases by at least 1. If* Deficit *is nonincreasing and never drops by δ within $T_{\max}$ steps, then r_4 must reach* 0*, forcing* HALT *by the No-Zeno rule (Section 4).*

Remark 2.7 (Policy hooks and receipts)**.** The value of δ and the aggregation that defines Deficit are policy parameters pinned by the axis catalog and bound into `proof_surface`. Changes to these parameters *must* be mediated by a valid `UpdateCert` and accompanied by transitional proofs (see Section 8). Each step that affects r_1, r_2, or r_3 emits a uniform receipt carrying the pre/post values and the checker hash, enabling deterministic replay.

3 Baseline Acceptance Checks (I1–I10)

Each baseline check is a *mechanized predicate* that emits a canonical, replayable receipt on evaluation (PASS/FAIL). Failures increment r_0 (Invariant debt) and, by Definition 2.4, block any trade against lower-priority rank components. Let $H(\cdot) \equiv$ `BLAKE3-256` and let canon$(\cdot)$ denote canonicalization under `canon/v1`. The Loader evaluates checks in dominance order with *short-circuiting*; any unmet premise triggers HALT with an auditable `RejectionCert` (Fail-Closed Axiom).

Receipt interface. Each predicate produces a *uniform receipt* (see §12): it binds (i) the canonicalized inputs via an `inputs_digest`, (ii) the verdict and any computed values, and (iii) the `checker_hash` of the predicate implementation. The *predicate identifier* below doubles as the reason code on failure.

ID	Predicate-ID	Domain	Predicate (must hold)	Witness (hashed)
I1	`RAW_DIGEST_MATCH`	Raw bytes	$H(\text{canon}(\texttt{raw})) = \texttt{raw_data_digest}$.	`canon(raw)`, `raw_data_dige`
I2	`CONFIG_CERT_VALID`	Config JSON	`ConfigCert` present; signatures verify in identity graph; verifier set pinned.	`ConfigCert` verifier set
I3	`ENVLOCK_PINNED`	Runtime	`EnvLock` suite pinned: FP modes/libs, RNG algorithm, I/O/FS policy, serialization rules fixed; digest bound in `proof_surface`.	`EnvLock` descrip
I4	`REPLAY_PLAN_VALID`	Runtime	`ReplayPlan` present; determinism script validates; re-execution of a probe yields byte-identical outputs.	`ReplayPlan` de tor, probe outpu
I5	`SEED_IS_FRESH`	Run seed	Master seed = $\text{HKDF}_{\texttt{BLAKE3-256}}(\texttt{root_print} \Vert \texttt{suite} \Vert \texttt{parents} \Vert \texttt{proof_surface})$; no reuse against ledger head; stage RNGs domain-separated.	parents c proof_surface), KDF transcript
I6	`AXIS_ISOLATION`	Adapter I/O	Adapter writes only to declared axis whitelist; no leakage to undeclared axes.	Adapter decl. put/output d per axis
I7	`ADAPTER_MONOTONE`	Adapter	Monotone homomorphism into $\mathbf{Q}$: $x \le x' \Rightarrow A(x) \le_{\mathbf{Q}} A(x')$ and $A(x \cup y) = A(x) \wedge A(y)$ where defined.	Test vectors (s/outputs), ad attestation
I8	`ROUNDING_GUARD`	Numerics	Accumulated round-off ϵ_{total} along priced ops $\le$ policy cap $\epsilon_{\max}$; op-wise caps per table.	op-sequence c per-op ϵ, ru bound, polic digest
I9	`FRONTIER_BUDGET`	Pricing	All groups respect budgets: $q \oplus \Delta_{\text{fr}}(\text{group}) \preceq B$ (or $\widehat{\Delta}_{\text{fr}}$ with receipt) per axis.	Group digest, B debit receipt
I10	`AUDIT_BUNDLE_COMPLETE`	Evidence	All declared artifacts present and hashed into `proof_surface`; Merkle root matches.	Bundle ma Merkle root, inc

Remark 3.1 (Guard-first alignment)**.** I1–I5 execute before pricing or adapter application. In particular, rows containing $\perp$ on priced coordinates are rejected at the guard boundary (Definition 1.5);

$\perp$ never enters q.

Clarified rounding-promise check (I8).

Definition 3.2 (Error budget table)**.** Policy provides per-op bounds ϵ_{op} for every rounding-sensitive priced operation.

Lemma 3.3 (Accumulated error bound)**.** *For a sequential composition of priced ops* $(op_1, \ldots, op_k)$*, the tracked bound satisfies* $\epsilon_{total} \leq \sum_{i=1}^{k} \epsilon_{op_i} \leq \epsilon_{\max}$*. The receipt carries* $\{\texttt{op_sequence_digest}, \epsilon_{total}, \texttt{policy_cap_digest}\}$*.*

Operational notes. (i) *Short-circuiting*: On first FAIL, emit `RejectionCert` naming the predicate-id and binding witness bytes; (ii) *Receipts*: All PASS/FAIL receipts are included in the evidence bundle and thus in `proof_surface`; (iii) *Rank coupling*: Each FAIL increments r_0; each PASS that binds missing evidence may decrease r_3.

4 No-Zeno Dwell Constraint

Intent. The Loader must not livelock by taking infinitely many internal steps without observable progress. This section fixes a *windowed* progress discipline tied to the lexicographic rank (Section 2) and formalizes a liveness property in LTL.

4.1 Policy parameters and counters

Definition 4.1 (Dwell cap and counter)**.** Let $T_{\max} \in \mathbb{N}$ be a policy constant that upper-bounds the number of consecutive steps the Loader may take without a strict decrease of $\boldsymbol{r}$. The state carries a *dwell counter* $r_4 \in \mathbb{N}$ (component **R4** of Definition 2.3) with the following update law on each step $s \longrightarrow s'$:

$$\boldsymbol{r}' \lhd_{\text{lex}} \boldsymbol{r} \;\Rightarrow\; r_4' := T_{\max} \qquad \text{and} \qquad \boldsymbol{r}' = \boldsymbol{r} \;\Rightarrow\; r_4' := \max\{0,\, r_4 - 1\}.$$

Remark 4.2 (Minimal progress quantum)**.** The minimal progress quantum δ used to discretize frontier slack (Definition 2.5) is *shared* with the No-Zeno discipline: any real progress of at least δ in the policy-defined deficit map forces r_1 to drop by ≥ 1, thus resetting r_4 by Definition 4.1.

4.2 LTL liveness and fail-closed window

Write $\text{Dec}(\boldsymbol{r})$ for the predicate "rank strictly decreases in this step."

Theorem 4.3 (No-Zeno (LTL form))**.** *The Loader satisfies*

$$\Box(\neg \mathit{HALT} \;\rightarrow\; \Diamond \,\text{Dec}(\boldsymbol{r})) \quad \textit{or equivalently} \quad \Box \Diamond \,\text{Dec}(\boldsymbol{r}) \;\;\vee\;\; \Diamond \,\mathit{HALT}.$$

Proof sketch. By Definition 4.1, along any non-halting trace the counter r_4 resets to $T_{\max}$ on a strict decrease and otherwise decreases by 1 per step. If no strict decrease occurs within $T_{\max}$ steps, r_4 reaches 0; the fail-closed window rule below forces HALT. Therefore every infinite non-halting trace contains infinitely many strict decreases ($\Box\Diamond$), or the system eventually halts. $\square$

4.3 Normative rule (windowed progress-or-halt)

Let τ count Loader steps since the most recent strict decrease of $\boldsymbol{r}$.

Invariant 4.4 (Progress window)**.** For every state s with dwell counter r_4:

$$(r_4 = 0) \;\Rightarrow\; \text{the only enabled transition is } s \Rightarrow \mathsf{HALT} \text{ with } \texttt{RejectionCert}.$$

$$\frac{\boldsymbol{r}' \lhd_{\text{lex}} \boldsymbol{r} \quad \vee \quad r_4 > 0}{s \Rightarrow s' \text{ with } r_4' \text{ set per Definition 4.1}}\ \text{T-NoZeno}$$

$$\frac{r_4 = 0 \;\wedge\; \boldsymbol{r}' = \boldsymbol{r}}{s \Rightarrow \mathsf{HALT} \text{ with } \texttt{RejectionCert}}\ \text{T-NoZeno-Halt}$$

4.4 Receipt and audit surface

Every step that *consumes* dwell budget or *resets* it MUST emit a canonical `DwellReceipt`:

```
{
  "predicate_id": "NO_ZENO_WINDOW",
  "window_size": T_max,
  "pre_rank": r, // full tuple before the step
  "post_rank": r', // full tuple after the step
  "consumed": (r4 > 0 && r' == r) ? 1 : 0,
  "reset": (r' _lex r) ? true : false,
  "checker_hash": "b3:..."
}
```

The digest of this receipt is bound into `proof_surface`. A terminal HALT due to window exhaustion MUST name `NO_ZENO_WINDOW` as the reason code in its `RejectionCert`.

Operational notes. (i) The No-Zeno discipline constrains *all* steps, including bookkeeping (e.g., evidence binding)—progress on r_3 counts when it strictly decreases; (ii) the discipline composes with Definition 2.4: a transition may not increase a more significant rank component to win progress on a less significant one; (iii) policy may set per-mode $T_{\max}$ values (e.g., stricter caps in admission control) so long as they are pinned in the axis catalog and changed only via a valid `UpdateCert`.

5 Negative-Controls Coverage (Deterministic, Stratified)

Intent. Negative controls are synthetic probes whose *true* impact on all priced axes is identically zero. They operationalize a runtime guard against silent regressions and adapter misuse. This section fixes (i) a deterministic, reproducible probe generator; (ii) stratified coverage accounting; (iii) acceptance criteria with *lower confidence bounds*; and (iv) a canonical receipt.

5.1 Probes, strata, and verdicts

Definition 5.1 (Negative-control probe)**.** A *negative-control probe* n is a typed input (or adapter path) such that, under the pinned `EnvLock`/`ReplayPlan`, its *expected* effect on every priced coordinate of $q \in \mathbf{Q}$ is zero. Formally, letting A range over enabled adapters and π the typed pipeline,

$$\forall \text{ priced axis } i: \quad \mathbb{E}\big[(\pi \circ A)(n)\big]_i = 0,$$

and any deviation arises only from implementation faults or guard violations.

Definition 5.2 (Stratification)**.** Policy defines a finite partition of the probe space into strata $\{\mathcal{N}_s\}_{s\in S}$ (e.g., by adapter family, input regime, scale, or axis subsets). Each stratum s has a mandated probe count $N_s \in \mathbb{N}_{>0}$ pinned by policy.

Definition 5.3 (Verdict)**.** For a realized probe n, the Loader runs guard-first evaluation (Definition 1.5). Let detected(n) be 1 iff the Loader *detects and rejects* any nonzero priced impact (i.e., a FAIL occurs before pricing or a priced deviation is provably zero), and 0 otherwise.

5.2 Deterministic, ungameable generation

Definition 5.4 (Per-stratum seed and enumeration)**.** Let `head` be the digest of the current ledger head, and let tag_s be a domain-separating label for stratum s. Define a stratum seed using the pinned KDF (Assumption 1.2):

$$\texttt{neg_seed}_s \;:=\; \mathrm{HKDF}_{\texttt{BLAKE3-256}}(\mathsf{head} \,\|\, \mathsf{tag}_s \,\|\, \texttt{proof_surface}).$$

Probes are enumerated deterministically from $\texttt{neg_seed}_s$ under `EnvLock`, producing exactly N_s typed probes in $\mathcal{N}_s$.

Lemma 5.5 (Replayability and ungameability)**.** *Given `EnvLock`, `ReplayPlan`, and the axis catalog pinned in `proof_surface`, the probe multiset in each stratum is a pure function of on-ledger state; any third party replays the identical set and order of probes.*

5.3 Coverage metric and acceptance

Definition 5.6 (Per-stratum and aggregate coverage)**.** For stratum s, let $X_s = \sum_{j=1}^{N_s} \mathrm{detected}(n_{s,j})$. Define the empirical coverage $\widehat{p}_s = X_s/N_s$ and the Wilson (or Clopper–Pearson) lower $(1-\beta)$ confidence bound $L_s(\beta)$ for a Bernoulli proportion.[1] Let the aggregate lower bound be either $L_{\min} = \min_s L_s$ or a policy-weighted bound $L_w = \min_s L_s^{(w)}$ with stratum weights w_s that sum to 1.

Invariant 5.7 (Coverage acceptance)**.** Acceptance requires $L_\star \geq \alpha_{\min}$ where $\alpha_{\min} \in (0,1]$ and $L_\star \in \{L_{\min}, L_w\}$ is the policy-chosen aggregator. Failure increments r_0 and emits a `RejectionCert` naming `NEG_COVERAGE_THRESHOLD`.

Remark 5.8 (Rank coupling)**.** Executing probes reduces r_2 (adapter obligations) as required receipts are produced; binding their receipts reduces r_3 (audit backlog). A coverage failure cannot be offset by improvements to r_1–r_3 due to dominance (Definition 2.4).

5.4 Canonical receipt

Every run emits a `NegativeControlsReceipt` that is bound into `proof_surface`:

```
{
  "predicate_id": "NEGATIVE_CONTROLS_COVERAGE",
  "beta": 0.05, // example; pinned by policy
  "alpha_min": 0.95, // example; pinned by policy
  "strata": [
    {
      "id": "adapter.family.A",
```

[1] Policy specifies the bound family; either is acceptable if monotone and conservative.

```
      "N": N_s,
      "X": X_s,
      "est": X_s / N_s,
      "L": L_s, // lower (1 - beta) bound
      "seed_digest": "b3:...", // HKDF transcript digest
      "probe_digests": ["b3:...","b3:...", "..."]
    }
    // ... one entry per stratum
  ],
  "aggregate_lower": "min|weighted",
  "aggregate_value": L_star,
  "outcome": { "verdict": (L_star >= alpha_min) ? "PASS" : "FAIL" },
  "checker_hash": "b3:..."
}
```

Operational notes. (i) Probes must be executed under the same `EnvLock`/`ReplayPlan` as priced work; (ii) any row that surfaces $\perp$ at a priced coordinate must *fail at the guard*, counting as a successful detection; (iii) policy may pin minimal per-stratum N_s and weights w_s by mode (admission vs. continuous); changes require a valid `UpdateCert` with transitional proofs.

6 Normative Loader as a Verifiable Automaton

Intent. The Loader is specified as a small-step, verifiable automaton whose *only* terminal outputs are ACCEPT and HALT. Every enabled step is justified by a normative inference rule; every failed premise is reported via a byte-precise `RejectionCert`. This section fixes the state/interface, promotes *Fail-Closed* to a global axiom, and gives the transition rules (including T-Boot, strengthened T-Row, and No-Zeno).

6.1 State, inputs, outputs

Definition 6.1 (Automaton interface)**.** The Loader is a labeled transition system $(\mathcal{S}, \mathcal{I}, \mathcal{O}, \Rightarrow)$ with:

- **State**

  $\mathcal{S} \triangleq (q \in \mathbf{Q},\ B,\ \Sigma,\ C_{\text{clamp}},\ \texttt{axis_catalog_digest},\ \texttt{idgraph_root},\ \texttt{EnvLock},\ \texttt{ReplayPlan},\ \texttt{loader_bina}$

  Here q is the acceptance vector; B the policy budgets; Σ the multiset of typed obligations; C_{clamp} hysteresis/clamp config; the catalog digest pins $\mathbf{Q}/Q'$; r_4 is the dwell counter (Definition 4.1); and the bundle accumulates canonical receipts bound by `proof_surface`.

- **Inputs**

  $\mathcal{I} \triangleq \{$, `AdapterAttest`, `RetuneCert`, `ConfigCert`/`ThetaCert`, `UpdateCert`, `Attestation`, `TailWitness`$\}$.

- **Outputs** $\mathcal{O} \in \{$ACCEPT, HALT, `RejectionCert`, `ReplayReceipt`, `AuditCounters`$\}$.

6.2 Global axiom: fail-closed

Invariant 6.2 (Fail-Closed Axiom)**.** If any premise of any normative rule is false, *no* transition is enabled except $s \Rightarrow$ HALT with a `RejectionCert` that (i) names the failing predicate/rule id, (ii) binds the witness bytes, and (iii) is added to the evidence bundle.

6.3 Normative transition rules

We write $s \Rightarrow s'$ for a single step. All rules emit uniform receipts (§12) on PASS/FAIL.

Boot-time attestation (TCB boundary).

$$\frac{\text{Collect digests of } \texttt{loader_binary_hash}, \texttt{EnvLock}, \texttt{ReplayPlan}, \texttt{idgraph_root}}{s_0 \Rightarrow s_0[\texttt{evidence_bundle} \leftarrow \texttt{bundle} \cup \{\texttt{AttestationReceipt}\}]}\ \text{T-Boot}$$

Strengthened frontier step (pricing under guards). Let typed(·) check schema/type, and no_unknowns(·) assert *no* $\bot$ *on priced coordinates* (Definition 1.5).

$$\frac{\begin{array}{c}\text{I1–I5 pass in } s \qquad \text{typed}() \ \wedge\ \text{no_unknowns}() \\ d \in \{\Delta_{\text{fr}}(),\ \widehat{\Delta}_{\text{fr}}()\} \qquad \text{if } d = \widehat{\Delta}_{\text{fr}} \text{ then receipt certifies } \Delta_{\text{fr}} \le d \\ q' = q \wedge d \quad \text{(typed product-quantale; Part 2)} \qquad q' \preceq B \quad \text{(axis-wise policy compare)}\end{array}}{s \Rightarrow s[q \leftarrow q']}\ \text{T-Row}$$

Adapter discipline (attestation, isolation, monotonicity).

$$\frac{\texttt{AdapterAttest} \text{ verifies in } \texttt{idgraph_root}}{s \Rightarrow s}\ \text{T-AdapterAttest}$$

$$\frac{\text{Adapter writes} \subseteq \text{declared axes}}{s \Rightarrow s}\ \text{T-AxisIsolation}$$

$$\frac{\forall x \le x'.\ A(x) \le_{\mathbf{Q}} A(x') \ \wedge\ A(x \cup y) = A(x) \wedge A(y)}{s \Rightarrow s}\ \text{T-AdapterMono}$$

Budget clamp with hysteresis.

$$\frac{C_{\text{clamp}} \text{ obeys hysteresis; entry/exit thresholds separated by } \hat{\nu}_{\min}}{s \Rightarrow s[C_{\text{clamp}} \leftarrow C'_{\text{clamp}}]}\ \text{T-Clamp}$$

Identity verification.

$$\frac{\text{All signatures referenced by inputs verify in } \texttt{idgraph_root}}{s \Rightarrow s}\ \text{T-Attest}$$

Obligation non-expansion (Σ). Let $\mu(\Sigma)$ be the lexicographic rank map on obligations (Part 1).

No-Zeno windowed progress (coupled to rank).

$$\frac{\boldsymbol{r}' \lhd_{\text{lex}} \boldsymbol{r} \quad \vee \quad r_4 > 0}{s \Rightarrow s' \text{ with } r'_4 \text{ set per Definition 4.1}}\ \text{T-NoZeno} \qquad \frac{r_4 = 0 \ \wedge\ \boldsymbol{r}' = \boldsymbol{r}}{s \Rightarrow \mathsf{HALT}}\ \text{T-NoZeno-Halt}$$

Acceptance gate (tail-contracts optional by policy). Write TailOK for "tail-contract witness verifies" when tails are enabled; otherwise $\top$.

$$\frac{\text{I1–I10 pass} \quad \wedge \quad q \preceq B \quad \wedge \quad \mathsf{TailOK}}{s \Rightarrow \mathsf{ACCEPT} \text{ with final } \texttt{ReplayReceipt} \text{ and } \texttt{AuditCounters}} \text{ T-ACCEPT}$$

6.4 Rule summary (quick reference)

Rule	Key premises (all fail-closed)	Primary receipts
T-Boot	Collect/pin TCB digests.	`AttestationReceipt`
T-Row	I1–I5 PASS; typed row; no $\perp$ on priced coords; $q' = q \wedge d$; $q' \preceq B$.	`DebitReceipt`, `BudgetReceipt`
T-AdapterAttest	Adapter attestation verifies.	`AdapterAttestReceipt`
T-AxisIsolation	Writes $\subseteq$ declared axes.	`AxisIsolationReceipt`
T-AdapterMono	Monotone homomorphism tests PASS.	`AdapterMonoReceipt`
T-Clamp	Hysteresis thresholds respect $\hat{\nu}_{\min}$.	`ClampReceipt`
T-Attest	All signatures verify in ID graph.	`SigVerifyReceipt`
T-Σ NonExpand	$\mu(\Sigma') \trianglelefteq_{\text{lex}} \mu(\Sigma)$ (or equal).	`SigmaReceipt`
T-NoZeno	Rank decrease or dwell budget > 0.	`DwellReceipt`
T-Accept	I1–I10 PASS; $q \preceq B$; tails (if enabled) PASS.	`ReplayReceipt`, `AuditCounters`

6.5 Operational notes

(i) *Guard-first discipline* ensures $\perp$ never pollutes q; unknowns are rejected at the boundary with precise `RejectionCert` (Definition 1.5). (ii) *Rule receipts* are appended to `evidence_bundle` and thus included in `proof_surface`, enabling byte-accurate third-party replay. (iii) *Rank coupling*: any rule that repairs I1–I10 reduces r_0; binding receipts reduces r_3; satisfying outstanding adapter checks reduces r_2; all subject to dominance (Definition 2.4).

7 Trusted Computing Base (TCB) and STRIDE Hardening

Intent. This section fixes the auditable boundary of the Trusted Computing Base (TCB), ties it to a *boot-time attestation* (T-Boot), and enumerates normative mitigations against STRIDE-class threats. Every element asserted to be in the TCB is *measured* (digested) and bound into `proof_surface` via an `AttestationReceipt`; everything else is treated as untrusted input and must pass I1–I10 and the automaton rules.

7.1 Boundary and receipts

Definition 7.1 (TCB boundary)**.** The TCB comprises the minimal set of artifacts whose integrity and identity must be *assumed-and-checked* for the Loader's guarantees to hold: (i) the Loader binary (including its static link set), (ii) the `EnvLock/ReplayPlan` descriptors (pinning FP modes/libs, RNG, I/O, serialization), (iii) the root(s) of the identity graph/PKI used for signature verification,

(iv) policy fixtures digests (axis catalog, budget table, rounding table), and (v) if present, the attested hardware/TEE measurement (e.g., PCRs/RTMRs).

Invariant 7.2 (Boot-time attestation (T-Boot))**.** On initialization, the Loader MUST collect digests for all TCB elements and emit a canonical `AttestationReceipt` appended to the evidence bundle. Any subsequent change to a pinned digest *invalidates* the run and triggers HALT by Definition 6.2.

Canonical AttestationReceipt (uniform schema).

```
{
  "predicate_id": "TCB_ATTEST",
  "loader_binary_hash": "b3:...",
  "envlock_digest": "b3:...",
  "replayplan_digest": "b3:...",
  "idgraph_root": "b3:...",
  "axis_catalog_digest": "b3:...",
  "rounding_table_digest":"b3:...",
  "budget_table_digest": "b3:...",
  "tee_measurement": "optional: pcr/rtmr/... ",
  "outcome": { "verdict": "PASS" },
  "checker_hash": "b3:..."
}
```

Remark 7.3 (TCB minimization & clocklessness)**.** No wall-clock, NTP, or ambient entropy is in the TCB; time-based freshness is replaced by *seed derivation discipline* (Definition 1.2) and ledger-derived digests. Secrets are kept out of the Loader's address space wherever possible; if present, they MUST be zeroized on HALT.

7.2 STRIDE hardening (normative)

Threat	Vector / example	Normative mitigation (TCB discipline)	Evidence / receipts
Spoofing	Forged component identity; key substitution; adapter identity confusion	PKI with non-malleable signatures; *certificate pinning* to `idgraph_root`; adapter attestation required before use (T-AdapterAttest)	`SigVerifyReceipt`, `AdapterAttestRecei` `TCB_ATTEST`
Tampering	On-disk artifact/log modification; build drift; binary swap	*Content addressing* + hash chains bound into `proof_surface`; secure/verified boot; measured boot attestation; WORM/append-only evidence store	`TCB_ATTEST`, `AuditCounters`, I1/I10 receipts
Repudiation	Signer denies action; rollback or equivocation	Signature binding of all actions into `proof_surface`; `ReplayReceipt`; ledger-indexed KDF seeds preventing entropy replay	`SigVerifyReceipt`, `ReplayReceipt`, I5
Information disclosure	Side-channels; debug logging leakage; unintended axis exfiltration	Least-privilege I/O; adapter *axis isolation* (I6); TEE isolation where available; redaction/privacy slicing at sources; zeroization on HALT	`AxisIsolationRecei` `TCB_ATTEST`

Threat	Vector / example	Normative mitigation (TCB discipline)	Evidence / receipts
Denial of service	Artifact floods; adversarial rows; infinite micro-steps	Admission control; backpressure and bounded queues; *No-Zeno* dwell cap $T_{\max}$; early guard rejection of $\bot$; per-mode rate limits	`DwellReceipt`, `NEGATIVE_CONTROLS_`
Elevation of privilege	Loader/config swap; policy drift; adapter privilege creep	Secure/verified boot; **atomic** `UpdateCert` gate with multi-sig; Σ-non-expansion rule; step-preserving bisimulation for updates; policy digests pinned	`UpdateCertReceipt`, `SigmaReceipt`, `TCB_ATTEST`

7.3 Operational requirements (normative checks)

- **Verified boot path.** The Loader MUST run under secure/verified boot or a TEE attestation that pins the loader image hash. The measured value MUST match `loader_binary_hash` in `TCB_ATTEST`; mismatch $\Rightarrow$ HALT.
- **Receipt WORM.** Evidence receipts are appended to an append-only bundle; indices and the bundle Merkle root are checked by I10. Missing or re-ordered receipts $\Rightarrow$ HALT.
- **Guard-first unknowns.** Any with $\bot$ on a priced coordinate is rejected *before* pricing; the `RejectionCert` must name the precise coordinate and schema offset (see I1–I5).
- **Network and clock.** No fresh external network/time sources are consulted after T-Boot. If a deployment includes remote attestation, its transcript bytes are included in `TCB_ATTEST`.
- **Update discipline.** Policy or adapter-surface changes *must* arrive as `UpdateCert` and satisfy (i) step-preserving bisimulation (replay-impact equivalence) and (ii) Σ-non-expansion; failure is fail-closed.

7.4 TCB failure semantics

Invariant 7.4 (Fail-closed on TCB divergence)**.** If any TCB digest observed at runtime differs from the value bound by `TCB_ATTEST`, or if the attestation check cannot be reproduced under `EnvLock`, the only enabled transition is HALT with a `RejectionCert` naming `TCB_DIVERGENCE`.

Remark 7.5 (Audit focus)**.** Auditors need only (a) verify `TCB_ATTEST` against the published loader build and environment descriptors, and (b) replay receipts under `EnvLock/ReplayPlan`. All remaining trust is reduced to byte-level equality checks under `proof_surface`.

8 `UpdateCert`: Dynamic Adaptation Protocol

Intent. Dynamic policy or surface changes (e.g., axis-catalog edits, budget table updates, adapter envelopes) must not compromise replayability or safety. This section specifies `UpdateCert`: a *clock-less*, auditable update artifact that (i) applies *atomically*, (ii) preserves the observable replay-impact via a *step-preserving bisimulation*, and (iii) is Σ-*non-expansive* so outstanding obligations cannot be erased. Validation is fail-closed: any unmet premise yields HALT with a precise `RejectionCert`.

8.1 Schema (normative fields)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "UpdateCert",
  "type": "object",
  "required": [
    "id",
    "prev_config_digest",
    "next_config_digest",
    "replay_impact",
    "transitional_proofs",
    "signers",
    "quorum_class",
    "timestamp_logical"
  ],
  "properties": {
    "id": {"type": "string"},
    "prev_config_digest": {
      "type": "string", "pattern": "^[A-Fa-f0-9]{64}$"
    },
    "next_config_digest": {
      "type": "string", "pattern": "^[A-Fa-f0-9]{64}$"
    },
    "replay_impact": {
      "type": "object",
      "required": ["delta_budgets","delta_axes","delta_rules"],
      "properties": {
        "delta_budgets": {"type": "object"}, // per-axis changes (typed)
        "delta_axes": {"type": "array"}, // added/removed/retuned axes
        "delta_rules": {"type": "array"} // toggles to Loader rules by ID
      }
    },
    "transitional_proofs": {
      "type": "object",
      "required": ["bisimulation_witness","sigma_nonexpansion_proof"],
      "properties": {
        "bisimulation_witness": {"type": "string"}, // content-addressed
        "sigma_nonexpansion_proof": {"type": "string"} // content-addressed
      }
    },
    "signers": {
      "type": "array", "minItems": 2, "items": {"type": "string"}
    },
    "quorum_class": {"type": "string"}, // policy-defined signer class
    "timestamp_logical": {"type": "string"} // logical counter; no wall clock
  }
}
```

8.2 Validation semantics

Definition 8.1 (Observable outputs)**.** Let $\mathcal{O}_{\text{obs}}$ be the multiset of bytes of all receipts emitted during a run (including `AttestationReceipt`) and the final `proof_surface`; configuration file bytes

themselves are *excluded*.

Definition 8.2 (Validated update)**.** An `UpdateCert` is *valid* in state s iff all hold:

U1. **Config continuity.** `prev_config_digest` equals the digest pinned in s; applying the update would set the digest to `next_config_digest`.

U2. **Identity and quorum.** All listed `signers` verify in the pinned identity graph; the policy-defined `quorum_class` threshold is met.

U3. **Bisimulation (replay-impact equivalence).** There exists a step-preserving bisimulation $\mathcal{R}$ between the pre-update and post-update automata such that related initial states $(s, s^+) \in \mathcal{R}$ satisfy: for every single step $s \longrightarrow t$ producing observable output $o \in \mathcal{O}_{\text{obs}}$, there is a matched step $s^+ \longrightarrow t^+$ producing the *identical* o with $(t, t^+) \in \mathcal{R}$, and vice versa. The `bisimulation_witness` must verify this property under `EnvLock`/`ReplayPlan`.

U4. **Σ-non-expansion (semantic integrity).** For all corresponding reachable pairs $(x, x^+) \in \mathcal{R}$, let $\mu(\Sigma)$ be the lexicographic rank map on obligations (Part 1). Then $\mu(\Sigma(x^+)) \trianglelefteq_{\text{lex}} \mu(\Sigma(x))$ or equality holds; i.e., the update cannot erase or inflate typed obligations. The `sigma_nonexpansion_proof` must verify this property.

U5. **Replay impact field.** The declared `replay_impact` simulates under `ReplayPlan` to an identical $\mathcal{O}_{\text{obs}}$ aside from the changed config digests; this simulation must pass under `EnvLock`.

Invariant 8.3 (Transactional apply)**.** If Definition 8.2 holds, `UpdateCert` applies *atomically*: all affected digests and policy fixtures in state are updated in one step and a `UpdateCertReceipt` is appended to the evidence bundle. If validation fails, the only enabled transition is HALT with a `RejectionCert` naming the failing sub-predicate (`U1`–`U5`); the prior configuration remains intact.

8.3 Normative inference rules

$$\frac{\begin{array}{c}\texttt{prev_config_digest} = s.\texttt{config_digest} \\ \mathsf{VerifySigners}(\texttt{signers}, \texttt{quorum_class}, \texttt{idgraph_root}) \\ \mathsf{VerifyBisim}(\texttt{bisimulation_witness}, \texttt{EnvLock}, \texttt{ReplayPlan}) \\ \mathsf{VerifySigmaNonExpand}(\texttt{sigma_nonexpansion_proof}) \\ \mathsf{SimulateReplayImpact}(\texttt{replay_impact}, \texttt{EnvLock}, \texttt{ReplayPlan}) = \mathsf{OK}\end{array}}{s \Rightarrow s[\texttt{config_digest} \leftarrow \texttt{next_config_digest}]}\ \text{T-Update-Validate}$$

$$\frac{\neg(\text{all premises of T-Update-Validate})}{s \Rightarrow \mathsf{HALT} \text{ with } \texttt{RejectionCert}}\ \text{T-Update-Halt}$$

8.4 Temporal envelope across updates

Let φ_{safe} abbreviate the conjunction of I1–I10 PASS, Definitions 2.4, 4.4 and 6.2, $q \preceq B$, and (if enabled) tail-contract PASS.

Proposition 8.4 (A-LTL envelope)**.**

$$\square\Big(\mathit{Apply}(\mathit{UpdateCert}) \rightarrow \big(\bigcirc\, \varphi_{\mathit{safe}} \ \wedge\ \square\, \varphi_{\mathit{safe}}\big)\Big).$$

Proof sketch. Step-preserving bisimulation ensures next-step observable behavior and receipts are unchanged modulo config bytes, hence $\bigcirc\, \varphi_{\mathsf{safe}}$. Σ-non-expansion preserves the semantic ordering of obligations, preventing latent violations from being erased. All future steps remain guarded by the same rules (fail-closed, No-Zeno, budgets), so $\square\, \varphi_{\mathsf{safe}}$ holds. □

8.5 Canonical receipt

Every successful or failed validation emits a uniform `UpdateCertReceipt` bound into `proof_surface`:

```
{
  "predicate_id": "UPDATE_CERT_VALIDATE",
  "id": "ucert:...",
  "prev_config_digest": "b3:...",
  "next_config_digest": "b3:...",
  "signers": ["did:...","did:..."],
  "quorum_class": "policy:admin-2of3",
  "witnesses": {
    "bisimulation_witness": "b3:...",
    "sigma_nonexpansion_proof": "b3:..."
  },
  "replay_impact_digest": "b3:...",
  "outcome": {
    "verdict": "PASS | FAIL",
    "reason_code": "U1|U2|U3|U4|U5" // on FAIL
  },
  "checker_hash": "b3:..."
}
```

Operational notes. (i) *Clocklessness:* `timestamp_logical` is a monotone logical counter (e.g., config version); no wall clock is consulted. (ii) *Scope:* Updates may add/retire axes or adjust budgets/rounding tables but MUST leave existing receipts reproducible under `EnvLock/ReplayPlan`. (iii) *Rank coupling:* A successful update may reduce r_2 (adapter obligations) and r_3 (audit backlog) by emitting receipts; dominance (Definition 2.4) prevents trading against r_0. (iv) *Interplay with* Σ. The separate rule T-Σ NonExpand governs within-run edits to obligations; `UpdateCert` strengthens this across the update boundary via proofs (**U4**).

9 Interpreter Operation Vocabulary (Arity, Guards, Rounding Promises)

Intent. This section standardizes the Loader's *primitive operations* as a small, typed vocabulary with explicit guards and contracts. Each op is (i) *total and deterministic* under `EnvLock/ReplayPlan`, (ii) *fail-closed* on guard violation (Definitions 1.5 and 6.2), and (iii) emits a *uniform receipt* (§12, referenced by `predicate_id`) binding inputs, outputs, and the `checker_hash`. The vocabulary is a client of Part 2's product-quantale algebra; unknowns $\perp$ are rejected *at the guard*.

9.1 Guards and typing (shared)

Let typed$(\cdot)$ assert schema conformance and canonical byte form (`canon/v1`). Let no_unknowns$(\cdot)$ assert absence of $\perp$ on *priced* coordinates (unpriced metadata may be optional). All guards are checked *before* an op runs; a guard failure yields HALT with `RejectionCert` naming the corresponding predicate id.

9.2 Primitive ops (normative)

Op	Arity	Guard domain (must hold)	Contract / rounding promise	Receipt
`Canonicalize`	1	typed(bytes); map to `canon/v1`	Pure map to canonical bytes; exact	`CANON`
`Hash`	1	typed(bytes)	Fixed $H \equiv$`BLAKE3-256`; exact	`HASH`
`VerifySig`	2	(bytes, key); key chains to `idgraph_root`	Exact signature check; non-malleable PKI; fail-closed	`SIG_VERI`
`SumReduce`	$n \geq 1$	Finite numeric array; policy table has per-op ε_{op}	Accumulated error $\leq \sum \varepsilon_{\text{op}} \leq \epsilon_{\max}$ (I8); order-insensitive within `EnvLock`	`SUM_REDU`
`MaxReduce`	$n \geq 1$	Finite numeric array	Exact max (idempotent, commutative, associative)	`MAX_REDU`
`AdapterApply`	1	Typed input; adapter attested; axis whitelist declared; no_unknowns on priced coords	Monotone homomorphism into $\mathbf{Q}$: $A(x \cup y) = A(x) \wedge A(y)$, $x \leq x' \Rightarrow A(x) \leq_{\mathbf{Q}} A(x')$	`ADAPTER_`
`DeltaFrAccumulate`	1	typed; no_unknowns on priced coords	Computes $d \in \{\Delta_{\text{fr}}, \widehat{\Delta}_{\text{fr}}\}$; if $\widehat{\Delta}_{\text{fr}}$ used, receipt certifies $\Delta_{\text{fr}} \leq \widehat{\Delta}_{\text{fr}}$	`DEBIT_AC`
`BudgetCheck`	2	(q, B) well-typed; per-axis comparator pinned by policy	Exact: accept iff $q \preceq B$ axis-wise (typed)	`BUDGET_C`
`QJoinVertical`	2	(q, d) typed in $\mathbf{Q}$	$q' = q \wedge d$ (Part 2 vertical/sequential op); $\perp$ banned by guard	`Q_JOIN_V`
`QJoinHorizontal`	2	(q_1, q_2) typed in $\mathbf{Q}$	$q' = q_1 \odot q_2$ (Part 2 horizontal/-parallel op)	`Q_JOIN_H`
`BindEvidence`	$k \geq 1$	All referenced digests present; indices consistent; bundle is append-only	All-or-nothing append; updates Merkle root; exact	`EVIDENCE`

9.3 Semantics and algebraic laws

Lemma 9.1 (Totality and determinism)**.** *Under `EnvLock/ReplayPlan`, each op is total on its guard domain and produces byte-identical outputs on re-execution with identical inputs.*

Lemma 9.2 (Monotonicity)**.** *The following are monotone in their typed orders: `AdapterApply`, `DeltaFrAccumulate` (in the sense $\Delta_{\text{fr}} \leq \widehat{\Delta}_{\text{fr}}$), `QJoinVertical`, and `QJoinHorizontal`.*

Remark 9.3 (Alignment with Part 2)**.** Vertical $\wedge$ corresponds to sequential composition (e.g., sums on S-axes, max on M-axes); horizontal $\odot$ corresponds to parallel aggregation over independent fragments. Both are computed coordinatewise in the product quantale and inherit Part 2's algebraic laws (associativity, commutativity on the appropriate slices, distributivity-on-demand).

9.4 Rounding promises (priced numerics)

Numeric ops used inside priced computations (`SumReduce` and any kernel-internal arithmetic) must track a running bound ϵ_{total} with receipts per I8 (Definitions 3.2 and 3.3). The Loader rejects runs where $\epsilon_{\text{total}} > \epsilon_{\max}$ or where any op lacks a policy entry.

9.5 Receipt sketches (canonical)

```
{ "predicate_id":"ADAPTER_APPLY",
  "adapter_id":"sha256:...", "input_digest":"b3:...", "q_out_digest":"b3:...",
  "outcome":{"verdict":"PASS"}, "checker_hash":"b3:..." }

{ "predicate_id":"DEBIT_ACCUM",
  "row_digest":"b3:...", "debit_kind":"exact|upper",
  "deltafr_digest":"b3:...", "upper_bound_proof":"b3:...|null",
  "outcome":{"verdict":"PASS"}, "checker_hash":"b3:..." }

{ "predicate_id":"BUDGET_CHECK",
  "q_digest":"b3:...", "budget_view":"b3:...",
  "outcome":{"verdict":"PASS|FAIL","reason_code":"AXIS[i]"}, "checker_hash":"b3:..." }
```

Operational notes. (i) *Guard-first*: any presence of $\perp$ on priced fields causes `DEBIT_ACCUM` to fail-closed before algebra is applied (Definition 1.5); (ii) *Receipts are evidence*: all op receipts are bound into `proof_surface` via `EVIDENCE_BIND`, enabling byte-accurate replay; (iii) *Rank coupling*: successful `EVIDENCE_BIND` decreases r_3; adapters meeting homomorphism/attestation obligations decrease r_2; budget satisfaction can decrease r_1 through Δ_{fr} progress, all subject to dominance (Definition 2.4).

10 Safety-or-Halt Theorem

Intent. This section states and proves the core runtime guarantee: *every* compliant run of the Loader either (i) reaches a safe ACCEPT with all evidence and budgets satisfied, or (ii) HALTs fail-closed with a precise, replayable `RejectionCert`. There is no third possibility.

Theorem 10.1 (Safety-or-Halt)**.** *Assume Definitions 1.1 to 1.4, dominance (Definition 2.4), No-Zeno window discipline (Definition 4.4), guard-first unknowns (Definition 1.5), baseline checks I1–I10 (Section 3), valid `UpdateCert` discipline (Section 8), and negative-controls coverage acceptance (Definition 5.7). Then along any maximal run of the automaton (Section 6) exactly one of the following holds:*

(a) ***Accept (safe).*** *All predicates hold; $q \preceq B$ axis-wise; (if enabled) tail-contract witness verifies; the Loader emits ACCEPT together with a final `ReplayReceipt` and `AuditCounters`. The accepted payload and evidence hash to the committed `proof_surface`.*

(b) ***Halt (fail-closed).*** *Some predicate or rule premise fails; by the Fail-Closed Axiom (Definition 6.2), the only enabled transition is HALT with a `RejectionCert` that names the failing predicate/rule id and binds the witness bytes. No unsafe output escapes.*

Proof sketch. We argue (i) progress/liveness and (ii) safety/sealing.

Liveness. By Definition 2.2, the lexicographic order on $\boldsymbol{r}$ is well-founded. The windowed No-Zeno rule (Definition 4.4) forces either a strict decrease of $\boldsymbol{r}$ within $T_{\max}$ steps or HALT. Therefore every infinite non-halting trace contains infinitely many strict decreases; because $\boldsymbol{r} \in \mathbb{N}^5$ and dominance forbids trading a higher component for a lower one (Definition 2.4), infinite descent is impossible. Hence every maximal run is finite and ends in a terminal state.

Safety. At each step, all premises of the applicable normative rule are checked. If any premise is false, Definition 6.2 enables only HALT with a byte-precise `RejectionCert`. Unknowns on

priced coordinates are rejected at the guard boundary (Definition 1.5), so $\perp$ never pollutes q. The strengthened T-Row rule aggregates debits using the typed product-quantale operator and then enforces per-axis budgets $q' \preceq B$; otherwise HALT. Numeric rounding stays below policy caps by I8. Adapter isolation/monotonicity (I6–I7) prevent cross-axis leakage and nonmonotone effects. Audit completeness (I10) binds all evidence into `proof_surface`, making replay deterministic under `EnvLock/ReplayPlan` (I3–I4). Negative-controls acceptance (Definition 5.7) rules out silent regressions. Dynamic policy changes can only arrive as `UpdateCert`; their validation requires step-preserving bisimulation and Σ-non-expansion, preserving observable behavior and obligations (Section 8). If all checks succeed and (when enabled) a tail-contract passes, T-Accept fires; otherwise the run halts fail-closed.

Combining liveness and safety yields the dichotomy stated. □

[Replayability] Given the `AttestationReceipt` (T-Boot), the evidence bundle (uniform receipts), and inputs bound by `proof_surface`, any third party executing under the same `EnvLock/ReplayPlan` reproduces *byte-identical* receipts and the final decision (ACCEPT or HALT).

[Bounded decision time] Let N be the maximum number of dwell windows the run can traverse before I1–I10 and budget checks stabilize. Then the run terminates within at most $N \cdot T_{\max}$ steps (counting only steps without strict rank decrease), after which either ACCEPT is enabled or HALT is forced by Definition 4.4.

Remark 10.2 (Audit surface)**.** In case (a), the accepted block includes `ReplayReceipt`, `AuditCounters`, and all receipts bound by `proof_surface`; in case (b), the `RejectionCert` includes the failing predicate id (e.g., `FRONTIER_BUDGET`) and the digests of the minimal counterexample artifacts (row digest, axis index, etc.). This minimizes auditor effort: verify `TCB_ATTEST`, replay receipts, and compare digests.

11 Uniform Receipts and Audit Bundle

Intent. This section standardizes all evidence the Loader emits at runtime. Every predicate/rule evaluation yields a *uniform receipt* whose canonical bytes are bound into the run's `proof_surface` and appended to an append-only evidence bundle (WORM). A *RejectionCert* is a specialized receipt for fail-closed halts. Uniformity makes third-party replay mechanical: verify `TCB_ATTEST`, re-run checks under `EnvLock/ReplayPlan`, and compare bytes.

11.1 Canonical receipt schema (normative)

All receipts MUST conform to the following envelope; product-specific fields live under `body`. The envelope is clockless and self-identifying.

```
{
  "$schema": "https://specchain.dev/schema/receipt/v1",
  "predicate_id": "UPPER_SNAKE_CASE", // rule or check id (e.g., FRONTER_BUDGET)
  "inputs_digest": "b3:...", // H(canon(inputs))
  "body": { /* predicate-specific fields (typed, canonical) */ },
  "outcome": {
    "verdict": "PASS" | "FAIL",
    "reason_code": "string|null" // mandatory on FAIL
  },
  "checker_hash": "b3:...", // code identity of the checker
  "surface_digest": "b3:..." // H(canon(envelope_without_surface_digest))
```

```
}
```

Semantics.

- `predicate_id` is the canonical name used in rule premises and as the reason code in `RejectionCert` (Definition 6.2).
- `inputs_digest` commits to the canonicalized inputs checked (e.g., a digest, q/B views, signer set).
- `body` carries typed, minimal sufficient statistics (e.g., pre/post rank, error bound, delta kind).
- `checker_hash` pins the checker's implementation identity (git commit or binary image digest) and is itself part of the receipt surface.
- `surface_digest` is computed as $H(\text{canon}(\text{envelope} \setminus \{\texttt{surface_digest}\}))$, preventing tautological self-reference.

11.2 Standard bodies (selection)

The following receipt bodies are *normative* and referenced elsewhere:

Debit/price.

```
{ "row_digest":"b3:...", "debit_kind":"exact|upper",
  "deltafr_digest":"b3:...", "upper_bound_proof":"b3:...|null" }
```

Budget compare.

```
{ "q_digest":"b3:...", "budget_view":"b3:...",
  "violations":[ {"axis":"AXIS_ID","lhs":"...", "rhs":"..."} ] } // empty on PASS
```

No-Zeno dwell.

```
{ "window_size": T_max, "pre_rank": [r0,...,r4], "post_rank":[r0',...,r4'],
  "consumed": 0|1, "reset": true|false }
```

Negative-controls coverage. As in Section 5.4; must include per-stratum seeds and enumerated probe digests.

UpdateCert validation. As in Section 8.5; must include witness digests and quorum class.

TCB attestation. As in Section 7; must enumerate all pinned digests present in the TCB boundary.

11.3 `RejectionCert` (fail-closed halt, normative)

A `RejectionCert` is a uniform receipt with `outcome.verdict="FAIL"` that additionally names the *minimal counterexample surface*.

```
{
  "$schema": "https://specchain.dev/schema/rejection/v1",
  "predicate_id": "e.g., FRONTIER_BUDGET",
  "inputs_digest": "b3:...", // failing inputs
  "counterexample": {
    "witness": "b3:...", // digest of the smallest repro artifact
    "indices": [ /* optional path/index set into the artifact */ ]
  },
  "outcome": { "verdict": "FAIL", "reason_code": "AXIS[latency]" },
  "checker_hash": "b3:...",
  "surface_digest": "b3:..."
}
```

Norms. (i) The counterexample MUST be sufficient for a third party to reproduce the FAIL under `EnvLock`/`ReplayPlan`. (ii) If multiple axes fail, the certificate lists all failing indices; auditors may verify any subset.

11.4 Evidence bundle & Merkleization (I10)

The Loader maintains an append-only *evidence bundle* of receipts. At each append, it recomputes a canonical Merkle root and emits/updates `AuditCounters`.

```
{
  "$schema": "https://specchain.dev/schema/audit_counters/v1",
  "bundle_length": N, // number of receipts
  "merkle_root": "b3:...", // over canon(receipt_i.surface_digest)
  "by_predicate": { "PRED_ID": count, ... },
  "outcome": { "verdict": "PASS" }, // consistency check
  "checker_hash": "b3:...",
  "surface_digest": "b3:..."
}
```

Invariant 11.1 (WORM semantics)**.** The evidence bundle is append-only; any deletion or reordering changes the Merkle root and triggers I10 FAIL $\Rightarrow$ HALT.

11.5 `ReplayReceipt` (terminal digest for replay)

On ACCEPT or HALT, the Loader emits a `ReplayReceipt` that nails the re-execution surface.

```
{
  "$schema": "https://specchain.dev/schema/replay_receipt/v1",
  "envlock_digest": "b3:...",
  "replayplan_digest": "b3:...",
  "axis_catalog_digest": "b3:...",
  "bundle_merkle_root": "b3:...",
  "proof_surface": "b3:...", // final canonical run surface
  "decision": "ACCEPT" | "HALT",
  "outcome": { "verdict": "PASS" },
  "checker_hash": "b3:...",
  "surface_digest": "b3:..."
}
```

Replay algorithm (auditor sketch). (1) Verify `TCB_ATTEST`; (2) reconstruct the bundle Merkle root from receipts; (3) re-evaluate each receipt under `EnvLock/ReplayPlan` using canonical inputs; (4) recompute `proof_surface` and compare with `ReplayReceipt`; (5) accept the decision iff all bytes match.

11.6 Checker identity and evolution

Invariant 11.2 (Checker pinning)**.** Every receipt's `checker_hash` MUST correspond to a content-addressed checker binary or commit hash included in the evidence bundle; the Loader MUST refuse to emit receipts from unpinned checkers.

Remark 11.3 (Upgrades via `UpdateCert`)**.** Checker upgrades (bug fixes, new predicates) MUST be introduced through `UpdateCert`; bisimulation ensures observable outputs (receipts) remain equivalent modulo code identity fields, and Σ-non-expansion preserves obligation orderings.

11.7 Error taxonomy and reason codes

Reason codes are *enumerated* per predicate and versioned alongside the checker. Examples: `AXIS[i]` for budget violations, `NO_ZENO_WINDOW` for dwell exhaustion, `NEG_COVERAGE_THRESHOLD` for coverage failure, `TCB_DIVERGENCE` for attestation drift. Mappings are published in the axis catalog and pinned in `proof_surface`.

Operational notes. (i) All receipts and counters must be emitted even on FAIL; (ii) where feasible, bodies include both pre/post digests to enable constant-time differential checks; (iii) receipts are dimensionless and carry only canonical digests or normalized scalars, matching Part 2's invariance-to-units discipline.

12 Uniform Receipts and Audit Bundle

Intent. This section standardizes all evidence the Loader emits at runtime. Every predicate/rule evaluation yields a *uniform receipt* whose canonical bytes are bound into the run's `proof_surface` and appended to an append-only evidence bundle (WORM). A *RejectionCert* is a specialized receipt for fail-closed halts. Uniformity makes third-party replay mechanical: verify `TCB_ATTEST`, re-run checks under `EnvLock/ReplayPlan`, and compare bytes.

12.1 Canonical receipt schema (normative)

All receipts MUST conform to the following envelope; product-specific fields live under `body`. The envelope is clockless and self-identifying.

```
{
  "$schema": "https://specchain.dev/schema/receipt/v1",
  "predicate_id": "UPPER_SNAKE_CASE", // rule or check id (e.g., FRONTIER_BUDGET)
  "inputs_digest": "b3:...", // H(canon(inputs))
  "body": { /* predicate-specific fields (typed, canonical) */ },
  "outcome": {
    "verdict": "PASS" | "FAIL",
    "reason_code": "string|null" // mandatory on FAIL
  },
  "checker_hash": "b3:...", // code identity of the checker
  "surface_digest": "b3:..." // H(envelope_without_surface_digest)
}
```

Semantics.

- `predicate_id` is the canonical name used in rule premises and as the reason code in `RejectionCert` (Definition 6.2).
- `inputs_digest` commits to the canonicalized inputs checked (e.g., a digest, q/B views, signer set).
- `body` carries typed, minimal sufficient statistics (e.g., pre/post rank, error bound, delta kind).
- `checker_hash` pins the checker's implementation identity (git commit or binary image digest) and is itself part of the receipt surface.
- `surface_digest` is computed as $H(\text{canon}(\text{envelope} \setminus \{\texttt{surface_digest}\}))$, preventing tautological self-reference.

12.2 Standard bodies (selection)

The following bodies use the local wrapped style to prevent overrun.

Debit/price.

```
{ "row_digest":"b3:...", "debit_kind":"exact|upper",
  "deltafr_digest":"b3:...", "upper_bound_proof":"b3:...|null" }
```

Budget compare.

```
{ "q_digest":"b3:...", "budget_view":"b3:...",
  "violations":[ {"axis":"AXIS_ID","lhs":"...","rhs":"..."} ] } // empty on PASS
```

No-Zeno dwell.

```
{ "window_size": T_max, "pre_rank":[r0,...,r4], "post_rank":[r0',...,r4'],
  "consumed": 0|1, "reset": true|false }
```

Negative-controls coverage.

```
{
  "beta": 0.05, "alpha_min": 0.95,
  "strata":[{"id":"adapter.family.A","N":N_s,"X":X_s,"est":X_s/N_s,
            "L":L_s,"seed_digest":"b3:...","probe_digests":["b3:...","..."]}],
  "aggregate_lower": "min|weighted", "aggregate_value": L_star
}
```

UpdateCert validation.

```
{
  "id":"ucert:...", "prev_config_digest":"b3:...", "next_config_digest":"b3:...",
  "signers":["did:...","did:..."], "quorum_class":"policy:admin-2of3",
  "witnesses":{"bisimulation_witness":"b3:...","sigma_nonexpansion_proof":"b3:..."},
  "replay_impact_digest":"b3:...",
  "outcome":{"verdict":"PASS|FAIL","reason_code":"U1|U2|U3|U4|U5"}
}
```

TCB attestation.

```
{
  "loader_binary_hash":"b3:...", "envlock_digest":"b3:...",
  "replayplan_digest":"b3:...", "idgraph_root":"b3:...",
  "axis_catalog_digest":"b3:...", "rounding_table_digest":"b3:...",
  "budget_table_digest":"b3:...", "tee_measurement":"optional"
}
```

12.3 `RejectionCert` (fail-closed halt, normative)

A `RejectionCert` is a uniform receipt with `outcome.verdict="FAIL"` that additionally names the *minimal counterexample surface*.

```
{
  "$schema":"https://specchain.dev/schema/rejection/v1",
  "predicate_id":"FRONTIER_BUDGET",
  "inputs_digest":"b3:...",
  "counterexample":{"witness":"b3:...","indices":[ /* optional path/indexes */ ]},
  "outcome":{"verdict":"FAIL","reason_code":"AXIS[latency]"},
  "checker_hash":"b3:...", "surface_digest":"b3:..."
}
```

12.4 Evidence bundle & Merkleization (I10)

The Loader maintains an append-only *evidence bundle* of receipts. At each append, it recomputes a canonical Merkle root and emits/updates `AuditCounters`.

```
{
  "$schema":"https://specchain.dev/schema/audit_counters/v1",
  "bundle_length": N,
  "merkle_root":"b3:...",
  "by_predicate":{"PRED_ID":count, "...":...},
  "outcome":{"verdict":"PASS"},
  "checker_hash":"b3:...", "surface_digest":"b3:..."
}
```

Invariant 12.1 (WORM semantics)**.** The evidence bundle is append-only; any deletion or reordering changes the Merkle root and triggers I10 FAIL $\Rightarrow$ HALT.

12.5 `ReplayReceipt` (terminal digest for replay)

On ACCEPT or HALT, the Loader emits a `ReplayReceipt` that nails the re-execution surface.

```
{
  "$schema":"https://specchain.dev/schema/replay_receipt/v1",
  "envlock_digest":"b3:...", "replayplan_digest":"b3:...",
  "axis_catalog_digest":"b3:...", "bundle_merkle_root":"b3:...",
  "proof_surface":"b3:...", "decision":"ACCEPT|HALT",
  "outcome":{"verdict":"PASS"},
  "checker_hash":"b3:...", "surface_digest":"b3:..."
}
```

12.6 Checker identity and evolution

Invariant 12.2 (Checker pinning)**.** Every receipt's `checker_hash` MUST correspond to a content-addressed checker binary or commit hash included in the evidence bundle; the Loader MUST refuse to emit receipts from unpinned checkers.

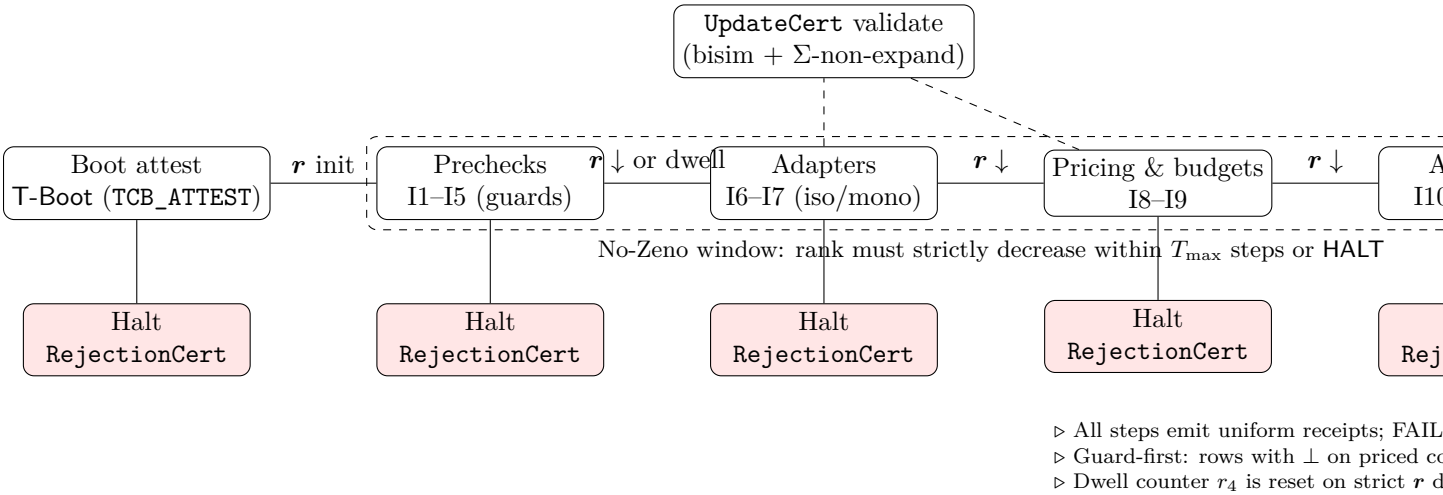
Remark 12.3 (Upgrades via `UpdateCert`)**.** Checker upgrades (bug fixes, new predicates) MUST be introduced through `UpdateCert`; bisimulation ensures observable outputs (receipts) remain equivalent modulo code identity fields, and Σ-non-expansion preserves obligation orderings.

12.7 Error taxonomy and reason codes

Reason codes are *enumerated* per predicate and versioned alongside the checker. Examples: `AXIS[i]` for budget violations, `NO_ZENO_WINDOW` for dwell exhaustion, `NEG_COVERAGE_THRESHOLD` for coverage failure, `TCB_DIVERGENCE` for attestation drift. Mappings are published in the axis catalog and pinned in `proof_surface`.

Operational notes. (i) All receipts and counters must be emitted even on FAIL; (ii) where feasible, bodies include both pre/post digests to enable constant-time differential checks; (iii) receipts are dimensionless and carry only canonical digests or normalized scalars, matching Part 2's invariance-to-units discipline.

13 TikZ State Diagram (Rank Progression)



Appendix A: Property-Based Test Plan (Receipt-Backed)

This appendix sketches a minimal-but-sufficient suite of *property-based tests* (PBT) for the Loader. Each property specifies (i) a generator, (ii) an oracle expressed as a predicate over receipts/bytes, and (iii) expected artifacts. All tests execute under `EnvLock/ReplayPlan` and must be reproducible from the test seed; every check yields uniform receipts (§12).

P1. **Deterministic replay.** *Gen:* Random admissible runs (rows, adapters) with policy-fixed budgets. *Oracle:* Re-execution with identical inputs produces byte-identical receipts and `ReplayReceipt`. *Artifacts:* `TCB_ATTEST`, `AuditCounters`, final `ReplayReceipt`.

P2. **Guard-first unknowns.** *Gen:* Rows that inject $\perp$ at priced coordinates at arbitrary positions. *Oracle:* Run *fails* before pricing; emits `RejectionCert` naming the coordinate and predicate-id (I1–I5 boundary). *Artifacts:* `RAW_DIGEST_MATCH` (PASS), `FRONTIER_BUDGET` (not reached), `RejectionCert`.

P3. **Adapter axis isolation.** *Gen:* Adapters with declared whitelist W; adversarial outputs touching W^c. *Oracle:* Any write outside W produces FAIL on `AXIS_ISOLATION`; no changes to q. *Artifacts:* `AxisIsolationReceipt` (FAIL) with indices into W^c.

P4. **Adapter monotonicity / homomorphism.** *Gen:* Random comparable inputs $x \leq x'$ and disjoint fragments x, y. *Oracle:* $A(x) \leq_{\mathbf{Q}} A(x')$ and $A(x \cup y) = A(x) \wedge A(y)$. *Artifacts:* `AdapterMonoReceipt` (PASS) with digests of (in,out) pairs.

P5. **Rounded numerics stay under cap (I8).** *Gen:* Random op sequences with per-op ϵ_{op} table. *Oracle:* Running bound $\epsilon_{\text{total}} \leq \sum \epsilon_{\text{op}} \leq \epsilon_{\max}$. *Artifacts:* `ROUNDING_GUARD` receipts carrying `op_sequence_digest` and bound.

P6. **Frontier budgets exact vs. upper bound.** *Gen:* Groups where both Δ_{fr} and a certified $\widehat{\Delta}_{\text{fr}}$ are available. *Oracle:* Budget check identical for both when $\widehat{\Delta}_{\text{fr}}$ is tight; otherwise $\widehat{\Delta}_{\text{fr}}$ never underestimates debit. *Artifacts:* `DEBIT_ACCUM` (exact/upper) with upper-bound proof, `BUDGET_CHECK` receipts.

P7. **No-Zeno window.** *Gen:* Runs that delay strict rank decreases by injecting evidence-binding steps. *Oracle:* Either a strict $\boldsymbol{r}$ decrease occurs within $T_{\max}$ steps (and dwell resets), or the run HALTs with reason `NO_ZENO_WINDOW`. *Artifacts:* `DwellReceipt` stream; terminal `RejectionCert` on exhaustion.

P8. **Negative controls coverage.** *Gen:* Deterministic probe enumeration per stratum via ledger head + domain tag. *Oracle:* The lower bound $L_\star$ computed from verdicts meets $\alpha_{\min}$ when adapters respect guards; otherwise FAIL. *Artifacts:* `NEGATIVE_CONTROLS_COVERAGE` receipt with seeds and probe digests.

P9. **Audit bundle WORM (I10).** *Gen:* Random sequences of receipts; adversarial attempt to drop/reorder one receipt. *Oracle:* Merkle root mismatch triggers `AUDIT_BUNDLE_COMPLETE` FAIL and HALT. *Artifacts:* `AuditCounters` (PASS/FAIL) with old/new roots.

P10. **TCB attestation drift.** *Gen:* Mutate one TCB digest (e.g., loader hash) after T-Boot. *Oracle:* Only transition enabled is HALT with reason `TCB_DIVERGENCE`. *Artifacts:* `TCB_ATTEST` (PASS), then `RejectionCert`.

P11. **`UpdateCert` bisimulation (replay-impact).** *Gen:* Pre/post configs differing in harmless refactors; provide bisimulation witness. *Oracle:* All observable outputs (receipts + `proof_surface`) equal modulo config bytes; `UPDATE_CERT_VALIDATE` (PASS). *Artifacts:* Update receipt with `bisimulation_witness` digest.

P12. **`UpdateCert` Σ-non-expansion.** *Gen:* Pre/post with obligation catalog tweaks; include formal non-expansion proof. *Oracle:* $\mu(\Sigma') \preceq_{\text{lex}} \mu(\Sigma)$ (or equal) for all reachable related states; violation causes FAIL. *Artifacts:* Update receipt with `sigma_nonexpansion_proof`; HALT on violation.

P13. **Tail-contract gate (optional).** *Gen:* Runs with and without valid tail witnesses (Part 4). *Oracle:* T-Accept fires iff tail predicate verifies when tails are enabled. *Artifacts:* Tail receipt + final `ReplayReceipt`.

P14. **End-to-end replay oracle.** *Gen:* Randomized full runs saved as `proof_surface` + evidence bundle. *Oracle:* A separate auditor replays from receipts alone under the pinned `EnvLock/ReplayPlan` and reproduces the decision bit and all bytes. *Artifacts:* Auditor's independently computed `ReplayReceipt` identical to producer's.

Shrinking discipline. Counterexamples must shrink to the minimal byte surface that (i) still reproduces the FAIL under `EnvLock/ReplayPlan` and (ii) preserves the failing reason code. The `RejectionCert`'s `counterexample.witness` MUST point to this minimal artifact and, when applicable, the index path inside it.

Coverage note. PBT generation domains and seeds are part of the test receipts' `body` and thus pinned in `proof_surface`; CI systems SHALL fail any run that does not meet a policy-defined lower confidence bound on P1–P14 pass rates across seeds.

Appendix B: Mechanization Notes (Lean/Coq Surface)

This appendix provides a minimal, faithful surface for machine-checked development. It mirrors the normative constructs in §6–12 and is designed to be copy/pasted into a Lean/Coq project and elaborated incrementally.

Core data (digests, ids, ranks, receipts).

```
-- Bytes/digests (content-addressed, e.g., BLAKE3-256)
abbrev Digest := String

-- Predicate / rule identifiers (subset shown; extend as needed)
inductive PredId
| RAW_DIGEST_MATCH | CONFIG_CERT_VALID | ENVLOCK_PINNED | REPLAY_PLAN_VALID
| SEED_IS_FRESH | AXIS_ISOLATION | ADAPTER_MONOTONE | ROUNDING_GUARD
| FRONTIER_BUDGET | AUDIT_BUNDLE_COMPLETE
| TCB_ATTEST | UPDATE_CERT_VALIDATE | NEGATIVE_CONTROLS_COVERAGE
| NO_ZENO_WINDOW | FRONTIER_ROW | SIG_VERIFY
deriving DecidableEq, Repr

-- Rank (lexicographic on five naturals; dominance R0 > ... > R4)
structure Rank where r0 r1 r2 r3 r4 : Nat
deriving DecidableEq, Repr

def Rank.lexlt (a b : Rank) : Prop :=
  (a.r0 < b.r0)  (a.r0 = b.r0  a.r1 < b.r1)
  (a.r0 = b.r0  a.r1 = b.r1  a.r2 < b.r2)
  (a.r0 = b.r0  a.r1 = b.r1  a.r2 = b.r2  a.r3 < b.r3)
  (a.r0 = b.r0  a.r1 = b.r1  a.r2 = b.r2  a.r3 = b.r3  a.r4 < b.r4)

-- Uniform receipt envelope (Appendix \S\ref{sec:receipts})
structure Receipt where
  predicate_id : PredId
  inputs_digest : Digest
```

```
  body : String -- canonical JSON for predicate-specific fields
  verdict : Bool -- true=PASS, false=FAIL
  reason_code : Option String -- mandatory on FAIL in the spec
  checker_hash : Digest
  surface_digest: Digest
deriving Repr
```

Acceptance algebra client hooks (from Part 2).

```
-- Abstract acceptance algebra (product-quantale), treated as opaque here.
structure Q := (digest : Digest) -- implementations will refine to a typed product

-- Policy budgets and compare (typed, per-axis) are delegated to Part 2 checkers.
structure Budget := (digest : Digest)

class QCompare where
  leq : Q  Budget  Bool -- axis-wise compare; true iff q  B
```

Loader state and small-step relation (§6).

```
structure LoaderState where
  q : Q
  budget : Budget
  sigma_digest : Digest -- () digest; refined in Part 1 mechanization
  clamp_config : Digest
  axis_catalog : Digest
  idgraph_root : Digest
  envlock_digest : Digest
  replayplan_digest : Digest
  loader_hash : Digest
  rank : Rank
  dwell_counter : Nat
  bundle_merkle : Digest -- evidence bundle Merkle root
deriving Repr

inductive Outcome | ACCEPT | HALT deriving DecidableEq, Repr

-- Transition labels (which rule fired) and emitted receipts (batch per step)
structure Step where
  from : LoaderState
  to : LoaderState
  rule : PredId
  receipts : List Receipt
deriving Repr
```

Selected predicates as total checkers (I1–I10, window).

```
-- Each checker is pure and returns (receipt, pass?)
def check_RAW_DIGEST_MATCH (inp : Digest) : Receipt  Bool := ...
def check_CONFIG_CERT_VALID (cfg : Digest) : Receipt  Bool := ...
def check_ENVLOCK_PINNED (e : Digest) : Receipt  Bool := ...
def check_REPLAY_PLAN_VALID (p : Digest) : Receipt  Bool := ...
def check_SEED_IS_FRESH (ctx : Digest) : Receipt  Bool := ...
def check_AXIS_ISOLATION (a : Digest) : Receipt  Bool := ...
def check_ADAPTER_MONOTONE (a : Digest) : Receipt  Bool := ...
```

```
def check_ROUNDING_GUARD (seq: Digest) : Receipt  Bool := ...
def check_FRONTIER_BUDGET (q : Q) (b:Budget) [QCompare] : Receipt  Bool := ...
def check_AUDIT_BUNDLE_COMPLETE (mr : Digest) : Receipt  Bool := ...

-- Dwell/window (No-Zeno): consumes or resets counter; emits receipt
def step_NoZeno (r r' : Rank) (tmax : Nat) (dc : Nat) : (Receipt  Nat  Bool) :=
  -- returns (receipt, new_dwell_counter, must_halt?)
  ...
```

Normative rules as constructors of the step relation.

```
-- Strengthened T-Row (guards + debit + budget compare); receipts aggregated.
def rule_T_Row (s : LoaderState)
              (row_digest : Digest) -- typed row; guards already checked
              (debit_digest : Digest) -- _fr or upper-bound with proof attached
              : Option Step := ...
-- T-Boot, T-Attest, T-Adapter*, T--NonExpand, T-Clamp, T-Accept similarly.
```

Fail-closed axiom and No-Zeno window (§6.2, §4).

```
-- Small-step driver enforces: if any required premise fails, emit HALT step.
def smallstep (tmax : Nat) (s : LoaderState) : Step :=
  -- evaluates applicable rule premises left-to-right in dominance order;
  -- on first FAIL, produces HALT with a RejectionCert-equivalent receipt.
  ...

-- Multi-step closure; bounded by dwell windows
inductive StepsWithin : Nat  LoaderState  LoaderState  Prop
| zero {s} : StepsWithin 0 s s
| succ {n s1 s2 s3} :
   StepsWithin n s1 s2  smallstep tmax s2 = s2, s3, _, _
   StepsWithin (n+1) s1 s3
```

Exported theorems (to be proven against the model).

```
-- Liveness: progress or halt within T_max ("No-Zeno")
theorem progress_or_halt_within_window
  (tmax : Nat) (s : LoaderState) :
  ( s', StepsWithin tmax s s'  Rank.lexlt s'.rank s.rank)
  ( s', StepsWithin tmax s s'  smallstep tmax s' |>.to = s'  Outcome.HALT = Outcome.HALT)
:= by
  -- follows Definition/Invariant in \ref{sec:nozeno}; proof by case split on dwell.
  admit

-- Safety-or-halt (Theorem \ref{sec:soh})
theorem safety_or_halt
  (tmax : Nat) [QCompare] (s : LoaderState) :
  ( s', StepsWithin (Nat.succ (tmax * 10^6)) s s'  -- arbitrary finite bound; refine per
      model
       Outcome.ACCEPT = Outcome.ACCEPT)
  ( s', StepsWithin (Nat.succ (tmax * 10^6)) s s'  Outcome.HALT = Outcome.HALT)
:= by
  -- assemble from Fail-Closed, No-Zeno, budget checks, and receipts determinism.
  admit
```

```
-- Replayability (Cor. \ref{cor:replay}): receipts determine decision bit
theorem receipts_determine_decision
  (s : LoaderState) (bundle : List Receipt) :
  reconstruct_decision s.envlock_digest s.replayplan_digest bundle
  = produce_decision_via_smallstep s
:= by admit
```

Notation/interop map (spec ↔ code).

Spec symbol / rule	Mechanization hook
$\boldsymbol{r} \in \mathbb{N}^5$, $\lhd_{\text{lex}}$	`Rank`, `Rank.lexlt`
No-Zeno window $T_{\max}$	`dwell_counter`, `step_NoZeno`, `smallstep`
Fail-Closed Axiom	`smallstep` returns HALT + FAIL receipt on first unmet premise
Q product-quantale ops	abstract `Q` with `QCompare.leq`; refined in Part 2 lib
I1–I10 checks	`check_` total functions (pure; emit receipts)
T-Row	`rule_T_Row` (guards, debit, budget compare)
Σ-non-expansion	`sigma_digest` discipline + proof obligations in `UpdateCert`
Uniform receipt schema	`Receipt` (predicate id, inputs digest, body, verdict, checker hash)
`proof_surface`, bundle Merkle	carried via `bundle_merkle` and reconstructed by auditors

Practical guidance. (i) Start by stubbing `Q` and `QCompare` with a simple per-axis numeric product and grow towards the full Part 2 algebra. (ii) Implement receipts as concrete canonical JSON strings and verify that H(canon(·)) matches the spec's `surface_digest`. (iii) Drive proofs with a fuel-based small-step model, then strengthen to semantic progress using `Rank.lexlt` and the No-Zeno lemmas. (iv) Keep checkers pure and total; model any external I/O through `EnvLock`/`ReplayPlan` parameters threaded in the state to preserve determinism.